

Propuesta de Proyecto — 1.ª Entrega

Título del proyecto

Sistema de Gestión y Trazabilidad Dietoterápica.

Integrantes

- Claria, Carolina
- Diaz, Natalia
- Herrera, Jazmin

Tutor

Juan Ignacio Schiavonni.

Tipo de aplicación

Web Full-Stack.

1. Introducción y Justificación del Problema

1.1. Introducción

En el Sanatorio “Unidos por la Salud (UxS)”, el Servicio de Alimentación y Dietoterapia cumple un rol asistencial y terapéutico crítico en la recuperación de los pacientes internados. Actualmente, el flujo operativo del área depende en su totalidad de soportes analógicos y planillas manuscritas en papel.

Relevamiento en sala: los licenciados en nutrición realizan los recorridos diarios sala por sala en los distintos pisos de internación del sanatorio, evaluando a cada paciente y registrando en planillas manuales los regímenes dietoterápicos correspondientes según su patología de base, consistencia tolerada o requerimientos clínicos específicos.

Traspaso a producción: dichas planillas físicas se entregan al sector de Cocina Central del sanatorio para que el personal de producción contabilice, elabore y fraccione las raciones requeridas para cada turno de comida (desayuno, almuerzo, merienda y cena).

Distribución final: con base en estos listados impresos o manuscritos, el cuerpo de camareros realiza el emplatado, rotulado y distribución directa de las bandejas a pie de cama.

1.2. Problemas detectados en el Sanatorio Unidos por la Salud (UxS)

Quiebre en la comunicación ante cambios dinámicos: si un paciente internado en el sanatorio sufre una descompensación, pasa a indicación de ayuno, recibe el alta médica o es trasladado de cama/piso con posterioridad al recorrido matutino, la planilla física entregada a cocina queda desactualizada. Esto provoca la elaboración innecesaria de raciones, desperdicio de insumos y traslados erróneos por parte de los camareros.

Riesgo en la seguridad del paciente por transcripción manual: la lectura manual de caligrafías apresuradas o tachaduras en papel incrementa el riesgo de servir preparaciones inadecuadas a pacientes con patologías estrictas (ej. diabéticos, celíacos, hipertensos o pacientes con disfagia).

Desarticulación entre preferencias y producción: el soporte analógico no permite cotejar de forma ágil si el menú del día contiene alimentos que el paciente rechaza o a los que presenta alergias/intolerancias, derivando en un alto índice de bandejas devueltas sin consumir y comprometiendo su adecuado estado nutricional.

Falta de trazabilidad y auditoría clínica: el registro en papel impide contar con un historial unificado donde consten las notas de evolución y la autoría formal de cada profesional que intervino en las indicaciones dietoterápicas.

1.3. Propuesta de solución

La implementación de una solución web integral adaptada al Sanatorio UxS optimizará los tiempos de respuesta entre el servicio de alimentación, el área de cocina y el equipo de camareros, garantizando la seguridad alimentaria y la trazabilidad clínica de cada paciente.

2. Objetivos del Proyecto

2.1. Objetivo General

Desarrollar una plataforma web integral para la digitalización y gestión del servicio de alimentación hospitalaria que permita la visualización de camas por sector/piso, la prescripción de regímenes dietoterápicos, el registro firmado de historias clínicas nutricionales y la validación automática de menús frente a preferencias y alergias de los pacientes.

2.2. Objetivos Específicos

- Implementar un panel visual e interactivo de mapeo hospitalario estructurado por pisos, sectores y estados de cama.
- Diseñar un módulo de gestión de historias clínicas nutricionales con registro inmutable de evoluciones, asociando cada nota a la firma y matrícula del profesional actuante.
- Parametrizar el catálogo de regímenes hospitalarios (General, Líquido, Blando, Blando con pollo, Hiposódico, Blando masticatorio, Diabético, Celíaco, Individual).
- Implementar un motor de matching (validación cruzada) que alerte al nutricionista en tiempo real cuando el menú preestablecido contenga ingredientes incompatibles con las aversiones o alergias del paciente.
- Generar consolidados de producción para el sector de cocina discriminados por turno (Desayuno, Almuerzo, Merienda, Cena), piso y tipo de régimen.
- Implementar autenticación y control de acceso basado en roles (Nutricionista, Personal de Cocina, Administrador).

3. Requerimientos del Sistema

3.1. Requerimientos Funcionales (RF)

RF01 - Autenticación y Perfil Profesional:

- Registro e inicio de sesión seguro de usuarios con asignación de roles.
- Perfil del nutricionista con almacenamiento de datos profesionales (nombre, apellido, matrícula nacional/provincial).

RF02 - Mapeo y Gestión de Camas:

- Visualización del mapa de camas por piso y sector, reflejando estados (Disponible, Ocupada, Mantenimiento, Bloqueada).
- Asignación y desasignación ágil de pacientes a camas activas.

RF03 - Historia Clínica Nutricional y Evolución:

- Registro de datos filiatorios y diagnósticos médicos de base del paciente.
- Carga de evoluciones nutricionales diarias/periódicas asociadas automáticamente al usuario autenticado (firma digital, matrícula y marca temporal).
- Consulta cronológica inmutable del historial de evoluciones y cambios de indicación.

RF04 - Preferencias, Alergias y Catálogo de Menús:

- Registro individual de preferencias clasificadas en: Aceptado, Aversión/Rechazo, Alergia e Intolerancia.
- Configuración de menús diarios/cíclicos desagregados por platos e ingredientes.

RF05 - Motor de Detección de Conflictos (Matching):

- Validación automática entre los ingredientes del menú asignado y el perfil de rechazos/alergias del paciente.
- Notificación visual inmediata ante incompatibilidades y habilitación de opciones de plato de reemplazo.

RF06 - Módulo de Cocina y Comandas:

- Generación de totales consolidados de raciones por turno y régimen para producción central.
- Emisión de rótulos/comandas individuales por cama con advertencias nutricionales visibles.

3.2. Requerimientos No Funcionales (RNF)

- RNF01 - Integridad Transaccional: cumplimiento de propiedades ACID para evitar solapamientos en la ocupación de camas o asignaciones duplicadas de dietas.
- RNF02 - Seguridad y Privacidad: encriptación de contraseñas.
- RNF03 - Usabilidad: interfaz reactiva, con código de colores estandarizado para regímenes y alertas de alérgenos.
- RNF04 - Arquitectura en Capas: desacoplamiento claro entre presentación, servicios de dominio, persistencia y base de datos.

4. Stack Tecnológico

A continuación se detallan las tecnologías elegidas para desarrollar el Trabajo Final Integrador, seguidas de su correspondiente justificación técnica.

Capa	Tecnología	Justificación técnica
Backend	Java + Spring Boot	Robustez empresarial, inyección de dependencias, tipado estático y facilidad para estructurar lógica de negocio compleja. Además, es la tecnología que el equipo ya domina, habiéndola utilizado en el TP2 de Programación IV con arquitectura en capas completa (entidades, repositorios, servicios, controladores, DTOs).
Seguridad	Spring Security + JWT	Control de acceso granular por roles (ROLE_NUTRICIONISTA, ROLE_COCINA, ROLE_ADMIN) y protección de endpoints REST.
Persistencia / ORM	Spring Data JPA (Hibernate)	Mapeo objeto-relacional eficiente, manejo de transacciones declarativas y simplificación de consultas. Tecnología ya utilizada por el equipo en trabajos previos (Programación III y IV).
Base de datos	PostgreSQL (relacional)	Motor relacional robusto que asegura integridad referencial entre camas, pacientes, historias clínicas y menús; datos con estructura estable y relaciones fuertes entre entidades.
Frontend	TypeScript + React (Vite)	Tipado estático de punta a punta, renderizado reactivo para la actualización del mapa de camas y modularidad de componentes. Es además la tecnología que el equipo cursa actualmente en Programación IV, reforzando su uso en paralelo al desarrollo del proyecto.
Estilos / UI	Tailwind CSS	Construcción de interfaces limpias, accesibles y responsivas.
Despliegue	Vercel (frontend) + Render (backend y base de datos)	Nivel gratuito, integración directa con GitHub para despliegue automático en cada actualización del repositorio, sin necesidad de gestión manual de servidor. Render permite alojar Java/Spring Boot y PostgreSQL en la misma plataforma.

4.1. Justificación del stack elegido

¿Por qué React para el frontend?

Se eligió porque es la tecnología que el equipo cursa actualmente en Programación IV, lo que permite reforzar su uso en paralelo al desarrollo del proyecto y reduce el costo de aprendizaje, ya que no se parte de cero. Además, para una aplicación web como esta, JavaScript/React es la opción natural del lado del cliente.

¿Por qué Java + Spring Boot para el backend?

El equipo ya domina esta tecnología: fue utilizada en el TP2 de Programación IV, donde se construyó una API REST completa con arquitectura en capas (entidades, repositorios, servicios, controladores, DTOs y manejo de excepciones). Trabajar con una tecnología conocida permite escribir y revisar código más rápidamente, reconocer errores comunes con mayor facilidad y conocer las limitaciones desde el inicio del proyecto. Spring Boot, además, es un framework maduro orientado a aplicaciones con lógica de negocio compleja y tipado seguro, adecuado para un sistema que maneja relaciones entre camas, historias clínicas, regímenes y menús.

¿Por qué base de datos relacional (SQL)?

Los datos del sistema tienen una estructura bien definida y estable (paciente, cama, historia clínica, régimen, menú), y las relaciones entre esas entidades son centrales al problema: un paciente ocupa una cama, tiene una historia clínica con evoluciones asociadas y un régimen dietoterápico vigente. Se requiere integridad referencial y consistencia transaccional, garantizadas de forma nativa por un modelo relacional.

¿Por qué Vercel (frontend) y Render (backend y base de datos)?

Ambas plataformas ofrecen un nivel gratuito, se integran directamente con GitHub para despliegue automático en cada actualización del repositorio, y no requieren gestión manual de servidor, lo cual es adecuado dado que el equipo no cuenta con presupuesto ni tiempo para administrar infraestructura propia. Render, además, permite alojar el backend en Java/Spring Boot y la base de datos PostgreSQL dentro de la misma plataforma, simplificando la configuración.

¿Tiene el equipo experiencia previa con estas tecnologías?

Sí, con Java, Spring Boot y bases de datos relacionales (JPA/Hibernate) el equipo ya trabajó en Programación III y IV. Con React se está comenzando a la par de la cursada de Programación IV, por lo que existe una curva de aprendizaje breve pero acompañada por la materia en simultáneo

De hacer antes de entregar

- Crear el repositorio de GitHub y agregar a los 3 integrantes como colaboradores.
- Estructurar el repositorio (carpetas /frontend, /backend, /docs, README.md).